A

Project Synopsis on

# “RECIPE HUB”

## Submitted to

R.T.M. Nagpur University, Nagpur
In partial fulfillment for the requirement of the degree of
Bachelor of Commerce in (Computer Application)
BCCA – III Year, Semester – VI

Department of Commerce & Management


Submitted by

“Nishant Singh”

“Manashay Chawre”

“Soumya Waghuke”

(BCCA – III Year, Semester – VI)


Guided by

“GEETA BHANDARKAR”

(Assistant Professor)

# G H Raisoni College of Arts, Commerce and Science, Nagpur

### An Autonomous Institution Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur Accredited by NAAC with "A" Grade

### 2025-26

## 1. Title of the Project

**Recipe Hub – A Web-Based Recipe Management System**

---

## 2. Objectives of the Project

**Digital Transformation:** To replace traditional paper cookbooks with a centralized, cloud-accessible repository for culinary knowledge.

**Intuitive Discovery:** To provide a seamless browsing experience categorized by meal types (Breakfast, Lunch, Dinner, Dessert).

**Precision Search:** To implement a robust search engine that allows users to find recipes by name or specific ingredients.

**Content Creation & Management:** To empower users to register, build profiles, and contribute their own recipes with high-quality imagery.

**Cloud-Native Storage:** To utilize Cloudinary for optimized image hosting, ensuring fast loading times and professional visual presentation

---

## 3. Project Category

The **Recipe Hub** is a full-stack web application designed using the **MERN (MongoDB, Express, React, Node.js)** architecture.

- **DBMS (MongoDB):** Unlike traditional RDBMS (like MySQL) that use fixed tables and rows, this project uses **MongoDB**, a NoSQL DBMS. It stores recipe data in flexible, JSON-like documents. This is ideal for recipes where the number of ingredients and steps varies significantly between entries.

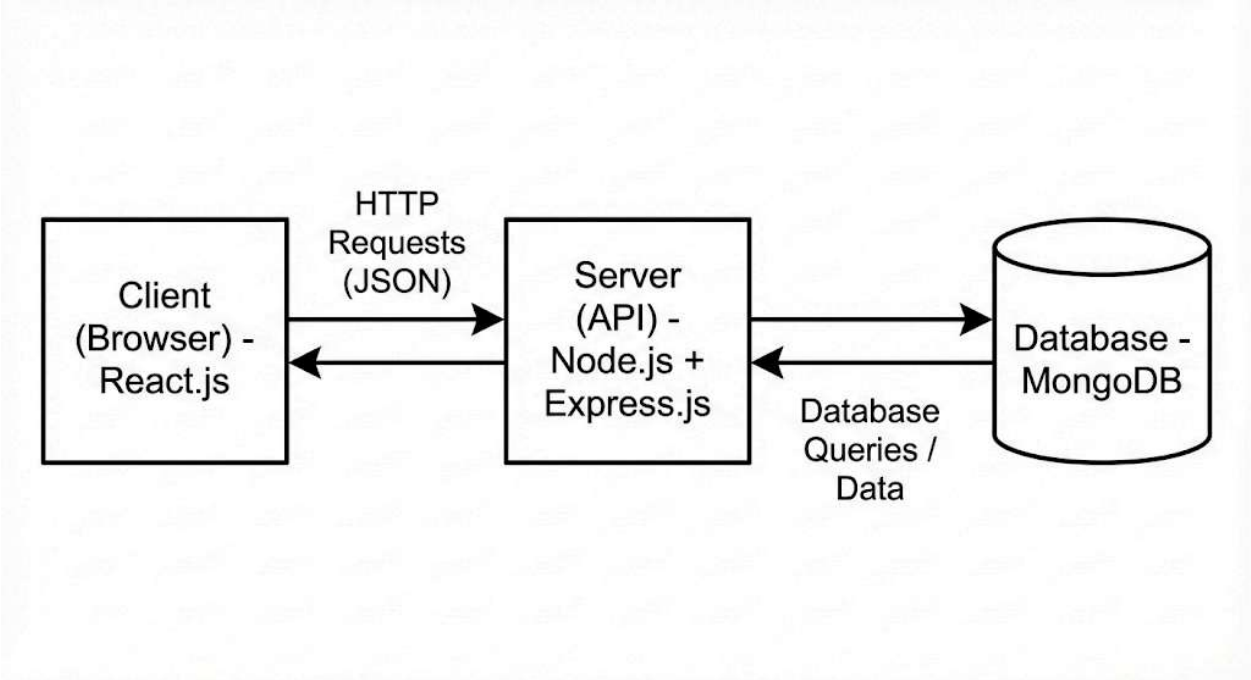
- **Application Logic:** The system follows the **Client-Server Architecture**. The React frontend communicates with the Node.js/Express backend via REST APIs to perform CRUD (Create, Read, Update, Delete) operations.
- **Media Management:** The system integrates **Cloudinary** as a cloud-based DBMS for images, ensuring the primary database remains lightweight and high-performing.

---

## 4. Tools / Platform and Languages to be used

This project will be built using the **MERN Stack**, which ensures a smooth workflow for both the front-end (what the user sees) and the back-end (where data is stored).

- **Front-End (Client Side):**
  - **React.js:** To build the user interface and pages.
  - **HTML/CSS:** For basic structure and styling.
- **Back-End (Server Side):**
  - **Node.js:** To run the server environment.
  - **Express.js:** To handle API requests (routing).
- **Database:**
  - **MongoDB:** To store user data and recipe details (NoSQL database).
- **Tools:**
  - **VS Code:** Code editor.
  - **Postman:** To test the APIs.




---

# 5. Complete Structure of the System

## I. Numbers of Modules and its Description

The **Recipe Hub** system is organized into three core functional modules that manage the flow of data from the user interface to the cloud storage and database.

### 1. User Authentication & Session Module

This module acts as the "Gatekeeper" of the application, ensuring data security and personalized user experiences.

- **Registration & Login:** Facilitates new user account creation and secure access for returning users.
- **Security (JWT):** Utilizes JSON Web Tokens to maintain the "Logged-in" state, allowing users to access private features like recipe uploading without re-entering credentials.
- **Profile Management:** Enables users to manage their identity and view the recipes they have personally contributed.

### 2. Recipe Management Module (CRUD)

The heart of the application, this module handles all interactions with recipe data and image hosting.

- **Create (Upload):** Users input recipe titles, ingredients, and steps. This sub-module triggers the **Cloudinary API** to upload images and retrieves a secure URL.
- **Read & Update:** Users can view their existing posts and make edits to ingredients or instructions if their cooking method changes.
- **Delete:** Provides users with the authority to remove their own recipes from the database.

### 3. Discovery & Filtering Module

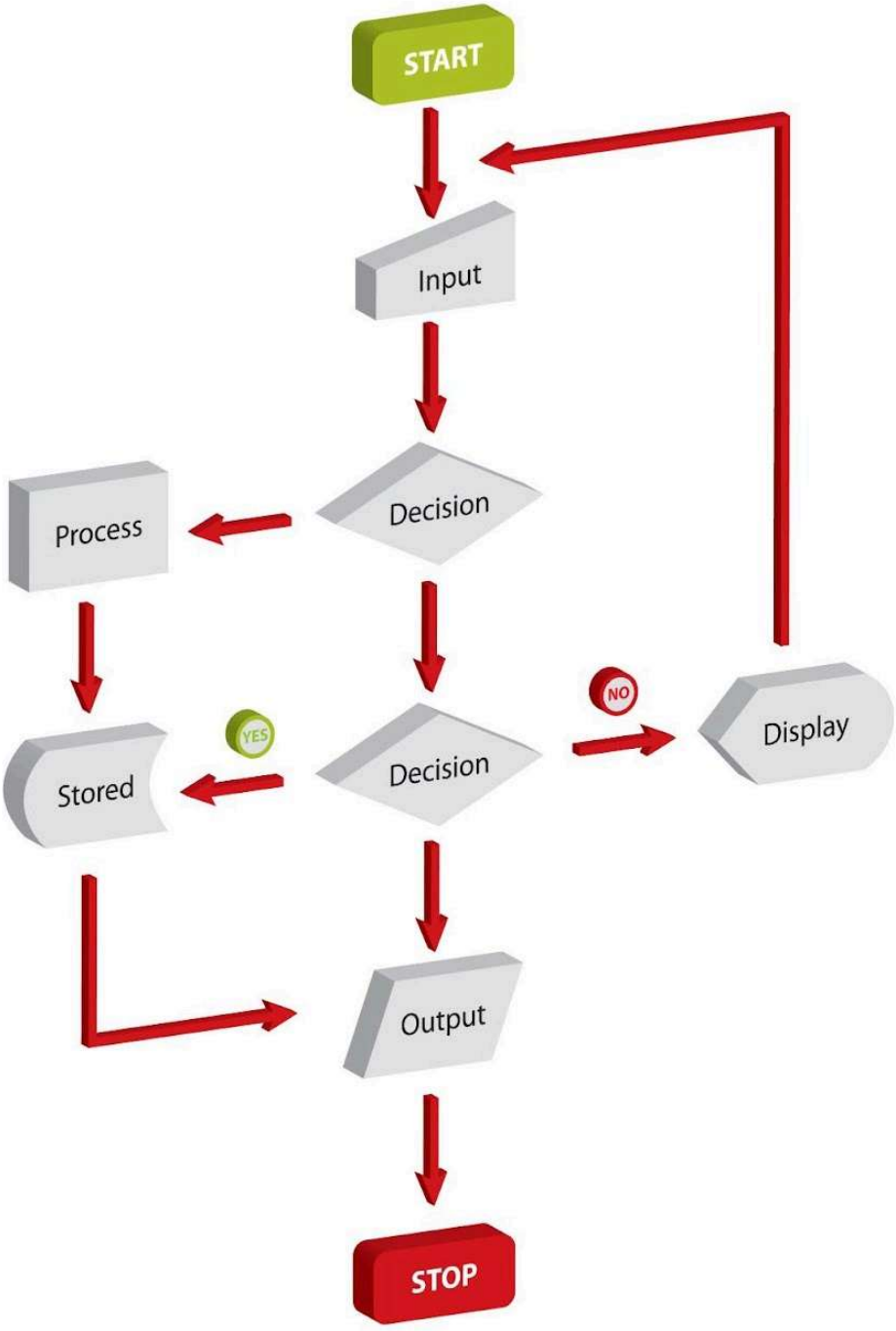
This module is responsible for how data is retrieved from MongoDB and presented to the public.

- **Dynamic Feed:** Automatically fetches the latest recipes and displays them on the Home Page using React components.
- **Categorization Logic:** A filtering system that sorts data based on meal types (e.g., Breakfast, Snacks) or dietary tags (Veg, Non-Veg).
- **Global Search:** A real-time search engine that queries the MongoDB database to find recipes matching specific keywords or ingredients.

## II. Modular Chart / System Chart

*(This represents the flow of the application)*

| Start | Action / Module | Description |
|---|---|---|
| ENTRY | Start | The user opens the website. |
| PUBLIC | Home Page | Users browse all recipes and watch videos |
| AUTH | Login / Register | Users sign in to gain contribution rights. |
| PRIVATE | Dashboard | The user's personal workspace opens. |
| ACTION | Add Recipe / Edit Profile | User uploads video, ingredients, and steps. |
| STORAGE | Database (MongoDB) | The system saves text and multimedia links. |
| OUTPUT | Home Page Update | The new recipe is pushed to the public feed. |


START
Input
Decision
Process
Stored
YES
Decision
NO
Display
Output
STOP

## III. Data Structures (MongoDB Collections)

In a NoSQL environment, we store data in flexible JSON-like documents. The **Users** collection is specifically designed to work with **Passport.js**.

### 1. Users Collection

- `_id`: Unique Primary Key.
- `username`: Unique display name.
- `email`: User's email (used as the unique username in Passport strategies).
- `password`: Hashed credentials (processed via **Bcrypt**).
- `role`: Defines user permissions (e.g., `user`, `admin`).

### 2. Recipes Collection

- `_id`: Unique identifier for the recipe.
- `title`: Name of the dish.
- `ingredients`: **Array** of strings.
- `instructions`: Detailed cooking steps.
- `category`: Tags such as Breakfast, Lunch, Veg, etc.
- `image_url`: Optimized hosting link from **Cloudinary**.
- `author_id`: **ObjectID** reference to the User collection (used for Authorization checks).

---

## IV. Process Logic

This section describes how **Passport.js** handles security and how recipes are created.

### 1. Authentication Logic (Login with Passport.js)

Passport.js uses a "Local Strategy" to verify users.

- **Step 1:** User submits email and password.
- **Step 2:** Passport-Local strategy intercepts the request and queries MongoDB for the email.
- **Step 3:** The system uses `bcrypt.compare` to check the password.
- **Step 4:** Upon success, Passport **serializes** the user into a session or generates a **JWT** for subsequent authorized requests.

### 2. Authorization Logic (Access Control)

This logic ensures users can only perform actions they are permitted to do.

- **Step 1:** When a user tries to Edit/Delete a recipe, the system checks the `author_id`.
- **Step 2:** It compares the `author_id` of the recipe with the ID of the currently logged-in user (from `req.user`).
- **Step 3:** If they match, access is granted; otherwise, a "403 Unauthorized" error is

returned.

### 3. Recipe Creation Logic (Add Recipe)

- **Step 1:** User uploads a photo; the frontend sends it to **Cloudinary**.
- **Step 2:** Cloudinary returns a secure URL; the frontend adds this URL to the recipe form.
- **Step 3:** The data is sent to the Node.js server where **Passport middleware** verifies the user is logged in.
- **Step 4:** The server performs a `.save()` to MongoDB, including the user's ID as the `author_id`.

---

## V. Types of Report Generation

These are dynamic data views generated by querying the database:

- **Category Inventory:** A live list of all recipes grouped by meal type.
- **User Portfolio:** A report of all recipes created by a specific user (Author Audit).
- **Access Report:** A list of users and their assigned roles (Authorization levels).

---

## VI. References


1. **React.js Documentation:** https://react.dev/
2. **MongoDB Manual:** https://www.mongodb.com/docs/
3. **Cloudinary Image Management:** https://cloudinary.com/documentation
4. **Passport.js Middleware:** https://www.passportjs.org/docs/
5. **Passport-Local Strategy:** http://www.passportjs.org/packages/passport-local/
6. **Bcrypt.js (Hashing):** https://www.npmjs.com/package/bcrypt
7. **Mongoose ODM:** https://mongoosejs.com/docs/


---